



III Межрегиональный Хакатон · UmirHack

Конструктор и центр управления реалистичными ловушками-приманками «HoneyForge»

Конфигурирование, развёртывание и мониторинг ловушек разного уровня.



1. Общие сведения о кейсе

Тема: Honeyrot (ловушки) - проектирование и разработка централизованной системы для массового управления ловушками-приманками.

Суть задачи. Командам предлагается разработать единую центральную платформу, через которую оператор (Blue Team / SOC) может: конфигурировать профили ловушек, разворачивать ловушки разного уровня взаимодействия на удалённых хостах, в реальном времени собирать с них всю входящую активность атакующих и исходящую телеметрию, а также хранить и анализировать эти данные. Ключевое требование - ловушки должны маскировать факт и канал связи с центром, чтобы атакующий не смог по сетевым признакам понять, что находится в ловушке.

2. Термины и определения

Термин	Определение
Honeyrot (ловушка)	Намеренно уязвимый или привлекательный для атакующего ресурс, единственная задача которого - быть атакованным, чтобы зафиксировать и изучить действия нарушителя. Легитимного трафика к нему быть не должно.
Уровень взаимодействия	Степень «реалистичности» ловушки: от эмуляции баннеров портов (low) до настоящей ОС (high). Определяет глубину собираемых данных и уровень риска.
Центр управления (C2 для ловушек)	Центральный сервер платформы: конфигуратор, оркестратор развёртывания, коллектор телеметрии, хранилище и UI.
Агент	Компонент на стороне ловушки, который принимает конфигурацию/команды от центра и отправляет собранную телеметрию, маскируя этот канал.
Профиль ловушки	Шаблон конфигурации: уровень, эмулируемые сервисы, баннеры, учётные данные-приманки, параметры логирования и маскировки.
Маскировка канала	Набор мер, скрывающих факт и содержимое связи ловушки с центром от атакующего, получившего доступ к ловушке или наблюдающего её трафик.
Honeytoken	Поддельный артефакт-приманка (ключ, учётка, файл, URL), срабатывание которого сигнализирует о компрометации.
Out-of-band (OOB) управление	Управление по отдельному сетевому пути/интерфейсу, недоступному из сегмента, где работает ловушка.

3. Контекст и актуальность

Классические ловушки разворачиваются и настраиваются поштучно, вручную, а собранные данные разрознены. При масштабировании до десятков ловушек это не работает: трудно поддерживать единые профили, централизованно собирать индикаторы компрометации и быстро реагировать. Вторая системная проблема - обнаружение ловушки атакующим. Продвинутый нарушитель, попав на хост, ищет признаки мониторинга: подозрительные исходящие соединения на «центр», нетипичные процессы-агенты, характерные артефакты. Если ловушка «спалена» - её ценность теряется.

Платформа из этого кейса решает обе проблемы: даёт единую точку управления парком ловушек и закладывает маскировку канала управления как архитектурный принцип.

4. Цель и задачи

Цель: спроектировать и реализовать MVP централизованной платформы управления парком honeyrot-ловушек разного уровня взаимодействия со скрытым каналом управления и сбором телеметрии.

Задачи команды

1. Спроектировать архитектуру: центр ↔ агенты ↔ хранилище ↔ UI.



2. Реализовать центральную систему: API управления, конфигуратор профилей, оркестратор развёртывания.
3. Реализовать агент ловушки, поддерживающий минимум два уровня взаимодействия (low и medium).
4. Обеспечить двусторонний обмен: конфиг/команды центр→ловушка и телеметрия ловушка→центр.
5. Реализовать базовую маскировку канала связи с центром.
6. Сделать dashboard: список ловушек, статусы, лента событий атак в реальном времени.
7. Подготовить сценарий демонстрации: атака на ловушку → событие в центре в реальном времени.

5. Общее описание платформы

Платформа состоит из трёх логических контуров, физически/сетевое разделённых:

- **Центр управления** - изолированный сервер: Control API (FastAPI), конфигуратор и оркестратор, коллектор телеметрии, PostgreSQL, realtime-слой (Node.js/WebSocket), Web-UI, фронт Nginx.
- **Парк ловушек** - хосты в изолированной сети-ловушке (DMZ): на каждом работает ловушка нужного уровня и скрытый агент.
- **Канал управления** - скрытый двусторонний канал между агентами и центром: шифрование, маскировка под легитимный трафик, по возможности отдельный mgmt-интерфейс.

Принцип разделения: атакующий взаимодействует только с сервисом-приманкой и не должен видеть ни центр, ни управляющий канал, ни контур mgmt.

6. Архитектура системы

Компоненты центра

Компонент	Назначение
Nginx (фронт)	TLS-терминация, reverse-proxy, разделение контуров, маскировка mgmt-API под обычный веб-ресурс.
Control API (FastAPI)	Управление профилями и ловушками, выдача конфигов, команды, выгрузка данных, аутентификация оператора.
Оркестратор	Рендер профиля в артефакт развёртывания (контейнер/скрипт) и доставка его на хост ловушки.
Коллектор телеметрии	Приём входных данных от агентов (события атак, сессии, артефакты), нормализация, запись в БД.
Realtime (Node.js)	WebSocket-поток событий в dashboard в реальном времени.
PostgreSQL	Хранилище: профили, ловушки, события, сессии, артефакты, honeytokens, аудит.
Web-UI	Dashboard оператора: парк ловушек, конструктор профилей, лента и аналитика событий.

7. Функциональные требования

Требования разделены на обязательные (Must) - минимум для зачёта кейса, и бонусные (Bonus) - для дополнительных баллов.

7.1 Центральная система

- **FR-C1 (Must).** CRUD профилей ловушек: уровень, набор сервисов/портов, баннеры, учётные данные-приманки, параметры логирования и маскировки.



- **FR-C2 (Must).** Регистрация ловушки в системе и привязка к профилю; просмотр списка ловушек со статусами (online/offline/last-seen).
- **FR-C3 (Must).** Выдача агенту конфигурации по профилю и приём от него телеметрии через защищённый endpoint.
- **FR-C4 (Must).** Хранение событий атак в PostgreSQL с возможностью фильтрации (по ловушке, времени, источнику, типу).
- **FR-C5 (Must).** Аутентификация оператора в UI/API (логин, токен/сессия, разграничение доступа).
- **FR-C6 (Bonus).** Оркестратор одной кнопкой: сгенерировать артефакт (Docker-образ/скрипт) и развернуть ловушку на удалённом хосте.
- **FR-C7 (Bonus).** Автоматические правила/алерты (напр. $\geq N$ попыток входа за минуту → уведомление) и экспорт IoC (CSV/STIX).

7.2 Агент и ловушка

- **FR-A1 (Must).** Поддержка минимум двух уровней взаимодействия (low и medium).
- **FR-A2 (Must).** Логирование активности: подключения, попытки аутентификации (вводимые логины/пароли), команды/запросы, сетевые метаданные источника.
- **FR-A3 (Must).** Периодическая отправка телеметрии в центр и получение обновлённой конфигурации.
- **FR-A4 (Must).** Устойчивость к потере связи: буферизация событий локально и досылка при восстановлении канала.
- **FR-A5 (Bonus).** High-interaction: реальная ОС в изолированной VM/контейнере с записью сессии (ввод/вывод, загруженные файлы).
- **FR-A6 (Bonus).** Honeytokens: раздача приманок (ключи/файлы/URL) и фиксация их срабатывания.

7.3 Данные: вход и выход

Входные данные (атакующий → ловушка → центр): IP/гео/ASN источника, timestamp, протокол/порт, введенные креды, выполненные команды/HTTP-запросы, payload'ы, загруженные файлы (хэш + артефакт), длительность и содержимое сессии.

Выходные данные (центр → ловушка): профиль/конфигурация, команды управления (старт/стоп/обновить/ротировать креды), параметры маскировки и расписание beacon'ов, набор honeytokens.

Выходные данные (центр → оператор): лента событий в реальном времени, агрегаты и статистика, карта/топ источников, IoC-экспорт, журнал аудита действий оператора.

8. Уровни ловушек (Low / Medium / High)

«Ловушки разного уровня» трактуются по уровню взаимодействия (interaction level) - это определяет глубину данных, сложность реализации и уровень риска.

Уровень	Что эмулирует	Собираемые данные	Риск / изоляция
Low	Открытые порты и баннеры сервисов (TCP/UDP), простые ответы. Без реальной логики сервиса.	Факт сканирования/подключения, порт, источник, баннер-запрос.	Минимальный. Процесс в контейнере.
Medium	Эмуляция логики сервисов: fake-SSH, fake-HTTP/админка, fake-БД. Отвечает как настоящий, но «внутри» - заглушка.	Креды, команды/запросы, навигация по fake-FS, payload'ы.	Умеренный. Контейнер без выхода в прод-сеть.
High	Реальная ОС/сервисы в строго изолированной VM/контейнере (sandbox).	Полная сессия, реальные действия, загруженные инструменты, lateral-попытки.	Высокий. Жёсткая изоляция, лимиты, авто-сброс (snapshot revert).



Для зачёта хватит рабочих Low и Medium.

9. Маскировка связи ловушки с центром

Цель - чтобы атакующий, оказавшийся на ловушке или наблюдающий её трафик, не распознал управляющий канал и не «спалил» ловушку. Рассматривается как инженерная задача скрытого управления инфраструктурой наблюдения.

Базовые требования (Must)

- **MASK-1.** Шифрование канала: TLS; трафик телеметрии не передаётся в открытом виде.
- **MASK-2.** Неприметность endpoint'a: приём телеметрии спрятан за обычным веб-ресурсом (путь/хост выглядят как легитимный сайт/CDN), а не как очевидный «/api/honeypot».
- **MASK-3.** Разделение интерфейсов: управляющий трафик по возможности идёт через отдельный mgmt-интерфейс/подсеть, недоступную из сегмента приманки.
- **MASK-4.** Скрытность агента на хосте: неприметные имя процесса/сервиса и расположение, отсутствие явных строк-маркеров «honeypot/C2».

Бонусные требования (Bonus)

- **MASK-5.** Джиттер и рандомизация beacon'ов: переменные интервалы и размеры запросов, чтобы не выделяться регулярностью.
- **MASK-6.** Анти-фингерпринтинг самой ловушки: реалистичные тайминги/баннеры/ошибки, устойчивость к типовым honeypot-детекторам.
- **MASK-7.** Взаимная аутентификация (mTLS) агента и центра, pinning, ротация ключей.

10. Нефункциональные требования

Категория	Требование
Безопасность центра	Центр изолирован от сегмента приманки; аутентификация и разграничение доступа; секреты не в коде (env/secret-store).
Изоляция ловушек	Ловушка не имеет доступа в продакшн-сеть; high-interaction - только в sandbox с лимитами ресурсов и авто-сбросом.
Надёжность	Потеря связи с центром не теряет данные (локальный буфер + досылка). Центр переживает рестарт без потери БД.
Масштабируемость	Горизонтальное добавление ловушек без переписывания центра; коллектор выдерживает пачки событий.
Производительность	Событие атаки доходит до dashboard в near-real-time.
Наблюдаемость	Логи компонентов центра, журнал аудита действий оператора.
Развёртываемость	Поднятие всего стека через docker-compose; воспроизводимость окружения.

11. Рекомендуемый технологический стек

Стек рекомендованный, допустимы замены.

Слой	Технологии
Backend / API	Python 3.11+, FastAPI
Realtime	Node.js (WebSocket) или нативные WebSocket FastAPI.
Хранилище	PostgreSQL. Опционально Redis (очереди/буфер)
Frontend	HTML/CSS/JS (можно лёгкий фреймворк); графики - Chart.js/аналог.



Слой	Технологии
Фронт/прокси	Nginx (TLS, reverse-proxy, маскировка mgmt-API).
Агент/ловушки	Python; эмуляторы сервисов; за основу допустимо брать open-source honeypot-движки.
Инфраструктура	Docker / docker-compose; VM-изоляция для high-interaction.

12. Критерии оценки

Работоспособность и целостность: Насколько решение действительно работает как единая система: центр, ловушки и канал связи между ними функционируют вместе, а не по отдельности. Ценится живая демонстрация, где видно, как активность на ловушке доходит до центра.

Глубина проработки темы: Насколько команда поняла суть задачи - зачем нужны ловушки разного уровня, какие данные с них важно собирать и почему маскировка связи критична.

Качество маскировки: Насколько убедительно ловушки скрывают связь с центром и насколько трудно их распознать.

Инженерная зрелость: Аккуратность архитектуры, адекватность выбранных решений, воспроизводимость запуска, читаемость кода и документации.

Подача и защита: Насколько ясно команда объясняет, что и зачем сделано, честно обозначает границы (что успели, что нет) и видит направление развития проекта.

13. Ожидаемые результаты

1. Git-репозиторий с исходным кодом и docker-compose для поднятия всего стека.
2. README: архитектура, инструкция запуска, описание профилей и API.
3. Работающий демо-стенд: центр + минимум 2 ловушки (Low и Medium).
4. Схема архитектуры и схема БД.
5. Короткая демонстрация сценария «атака → событие в центре».
6. Презентация защиты: решение, что успели, чем маскируются, планы развития, демонстрация.

Контакты Экспертов:

Захаров Данил - @syharik3316

Гринько Елена - @kdjirl